

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1714

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.*
(BTL 5)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour
Total Marks:50

Design Task

Code of Conduct:

*I **Priyanka.G / 192524124** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Design Task

Edge AI System for Industrial Quality Control

Design an Edge AI system integrated with a PySpark-based backend to inspect products on a high-speed manufacturing line using RDD operations for batch analysis of historical defect data. Explain how distributed data processing supports large-scale model retraining, and describe how intelligent agents at the edge collaborate with the central system to detect defects in real time and adapt inspection thresholds as production conditions change.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations /actions	Mostly correct usage	Limited or partial use	Incorrect/missing

Design Task: Edge AI System for Industrial Quality Control

Design an Edge AI system integrated with a PySpark-based backend to inspect products on a high-speed manufacturing line using RDD operations for batch analysis of historical defect data. The solution explains how distributed data processing supports large-scale model retraining, and describes how intelligent agents at the edge collaborate with the central system to detect defects in real time and adapt inspection thresholds as production conditions change.

1. Problem Understanding & Requirement Analysis

1.1 Problem Statement

A high-speed manufacturing line produces thousands of units per hour. Manual or purely cloud-based visual inspection cannot meet the millisecond-level latency required to reject defective items before they move past the inspection station. At the same time, the factory accumulates large volumes of historical defect-image and sensor data that must be analysed in batch to continuously improve the defect-detection model and to understand long-term quality trends across shifts, machines, and product lines. This mirrors the deployment challenge identified by Islam et al. (2024), who note that industrial defect-detection systems must jointly satisfy strict inference-latency budgets and continuous large-scale retraining from accumulated production data.

1.2 Functional Requirements

- Real-time (sub-100 ms) defect classification for each item on the conveyor using an edge-deployed model.
- Batch analytics over historical inspection logs, sensor readings, and rejected-item images stored in a data lake.
- Periodic and event-triggered retraining of the detection model using distributed compute (PySpark).
- Autonomous adaptation of inspection thresholds when drift, material change, or lighting conditions are detected.
- Two-way communication between edge agents and central coordination/orchestration service.

1.3 Non-Functional Requirements

- Scalability: the architecture must scale from a single line to a multi-plant deployment.
- Fault tolerance: loss of network connectivity to the cloud must not stop line inspection.
- Low latency at the edge; high throughput in the batch/analytics layer.
- Security: encrypted model updates and authenticated agent-to-cloud communication.

1.4 Constraints

- Limited compute and memory on edge devices (industrial PCs / NVIDIA Jetson-class hardware).
- Intermittent or bandwidth-limited connectivity between shop floor and central data centre / cloud.
- Regulatory requirement to retain inspection records for traceability and audits.

2. System Architecture Design

The system follows a three-tier Edge–Fog–Cloud architecture that separates real-time inference from heavy distributed analytics, while keeping the two tiers tightly coupled through a feedback loop. This layered separation follows established practice in recent edge-computing literature for industrial IoT, where lightweight inference is pushed to the device layer while historical analytics and retraining remain centralised for scalability (Lu et al., 2023).

2.1 Architectural Layers

Layer 1 Edge Layer (on the production line)

- High-speed industrial cameras and sensors (vibration, temperature, dimension) capture data per unit.
- An Edge Inference Agent (lightweight CNN, e.g., quantized MobileNet/YOLO variant, following the accuracy–efficiency trade-offs demonstrated for edge deployment by Qin et al., 2022) performs real-time defect classification directly on edge hardware (Jetson/industrial PC).
- A Local Decision & Actuation module triggers the reject mechanism (PLC/robotic arm) within milliseconds.
- A local buffer/cache stores raw frames and metadata for later upload, ensuring the line keeps running even if the network is down.

Layer 2 Fog/Gateway Layer (plant-level)

- A plant-level gateway aggregates data from multiple edge agents/lines.
- Performs data compression, deduplication, and pre-processing before forwarding to the cloud.
- Hosts a lightweight Kafka/MQTT broker for streaming telemetry and model-update messages between edge and cloud.

Layer 3 Cloud/Central Layer (data centre)

- A distributed data lake (HDFS / S3-compatible storage) stores raw and processed inspection data.
- Apache Spark (PySpark) cluster performs RDD-based batch analytics and periodic large-scale model retraining, consistent with Spark-based predictive-maintenance platforms proposed for smart manufacturing (Su et al., 2024).
- A Central Coordination Agent monitors all edge agents, aggregates drift signals, and pushes updated models/thresholds back to the edge.
- A model registry (e.g., MLflow) versions and validates each retrained model before deployment.
- A monitoring and alerting dashboard (e.g., Grafana/Prometheus) provides real-time visibility into cluster health, model performance, and drift metrics across all edge agents.
- An access-controlled audit log records all inspection data, model versions, and threshold changes to satisfy regulatory traceability requirements for quality and compliance audits.

2.2 High-Level Architecture Diagram

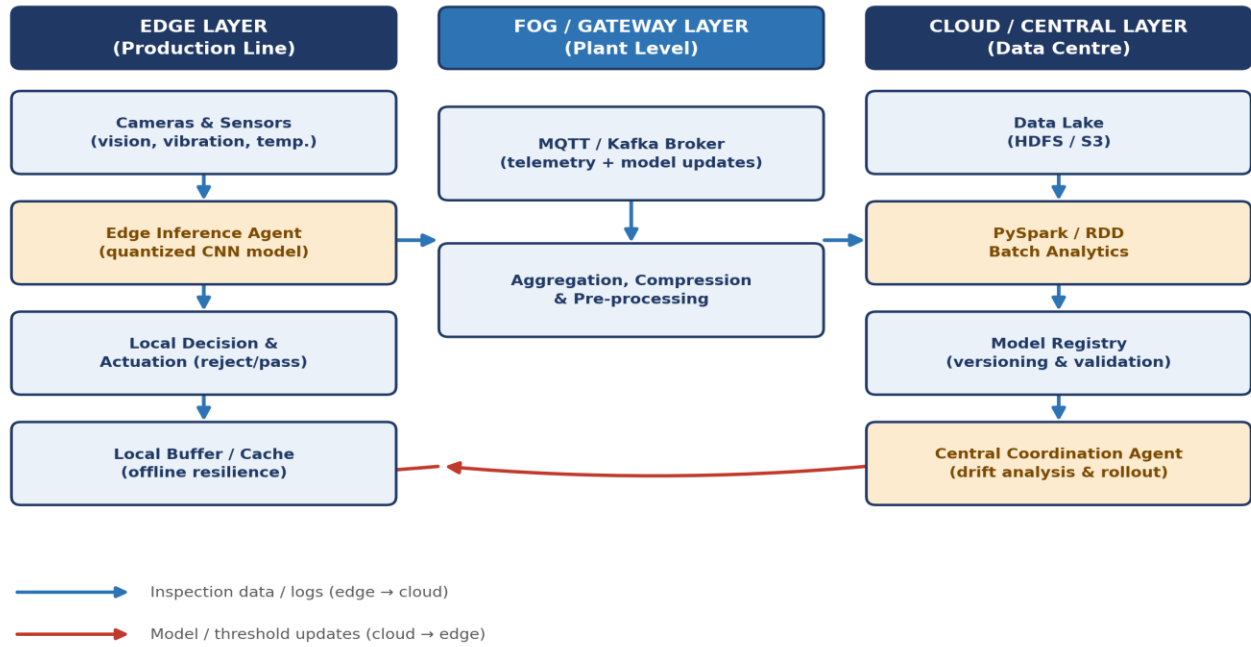


Figure 1. Three-tier Edge–Fog–Cloud system architecture showing forward dataflow (blue) and backward model/threshold feedback (red).

2.3 High-Level Architecture (textual summary)

EDGE LAYER	FOG / GATEWAY LAYER	CLOUD / CENTRAL LAYER
Cameras + Sensors → Edge Inference Agent → Local Actuation (reject/pass) → Local Buffer	MQTT/Kafka Broker → Aggregation → Compression → Forwarding	Data Lake (HDFS/S3) → PySpark/RDD Batch Analytics → Model Registry → Central Coordination Agent

Data flows left-to-right for inspection results and historical logs; model updates, retrained weights, and revised thresholds flow right-to-left, closing the feedback loop between the cloud analytics layer and the edge agents.

3. Use of PySpark & RDD Operations

PySpark forms the batch-analytics backbone of the system. RDDs (Resilient Distributed Datasets) are used because historical defect data — images metadata, sensor logs, and inspection results — is large, semi-structured, and needs fault-tolerant distributed processing across a commodity cluster. This choice is consistent with prior Spark-based industrial platforms, in which RDD-backed pipelines were shown to support scalable ingestion and machine-learning preparation directly from time-series sensor and quality data (Su et al., 2024).

3.1 Ingestion

Raw inspection logs (JSON/Parquet) from every edge agent are ingested into HDFS and loaded as RDDs: `sc.textFile('hdfs:///quality/logs/*')` for schema-flexible pre-processing, before conversion to DataFrames for downstream SQL-style analytics.

3.2 Representative RDD Transformations & Actions

- `map()` parse each raw JSON record into a structured tuple of (timestamp, line_id, defect_type, confidence, sensor_readings).
- `filter()` retain only records where confidence < threshold or defect_type != 'none', isolating uncertain or defective cases for retraining.
- `flatMap()` expand multi-defect records (an item can have more than one defect) into individual defect events.
- `reduceByKey()` aggregate defect counts by (line_id, defect_type) to compute defect-rate-per-line-per-shift.
- `groupByKey()` / `combineByKey()` group historical samples by defect class to build balanced training batches.
- `mapPartitions()` apply heavier per-partition operations (e.g., image feature extraction) efficiently, avoiding per-record function-call overhead.
- `sample()` draw stratified samples of historical data for validation-set creation.
- `persist()/cache()` cache the cleaned RDD in memory across iterative retraining passes to avoid recomputation.
- `collect()/count()/take()` actions used to pull aggregated summaries (not raw big data) back to the driver for dashboards and reports.

3.3 Why RDDs (and not only DataFrames) here

- Fine-grained control over partitioning is needed to balance defect classes across the cluster before training.
- Custom Python UDF-heavy transformations (image decoding, feature extraction) are more naturally expressed as RDD `map/mapPartitions` operations.
- Fault tolerance via lineage graphs allows automatic recovery of a partition if a worker node fails during a long retraining job important for multi-hour batch jobs on factory data.

3.4 Illustrative PySpark Implementation

The listing below sketches the core batch job that ingests raw inspection logs, isolates uncertain/defective samples, computes per-line defect rates, and prepares a class-balanced sample for retraining. It is representative of the RDD operations described in Sections 3.1–3.2 and would run as a scheduled or drift-triggered job on the cloud Spark cluster shown in Figure 1.

```
# -- PySpark batch job: defect-log analytics & retraining-set preparation ---
from pyspark import SparkContext, SparkConf
import json

conf = SparkConf().setAppName("QualityControl-RDD-Batch")
sc = SparkContext(conf=conf)

# 1. Ingest raw JSON inspection logs from the data lake (HDFS/S3)
raw_rdd = sc.textFile("hdfs:///quality/logs/*.json")

# 2. map(): parse each record into a structured tuple
def parse_record(line):
    r = json.loads(line)
    return (r["line_id"], r["defect_type"], r["confidence"], r["timestamp"])

parsed_rdd = raw_rdd.map(parse_record)

# 3. filter(): keep only uncertain or defective samples for retraining
CONFIDENCE_THRESHOLD = 0.85
```

```

candidate_rdd = parsed_rdd.filter(
    lambda x: x[2] < CONFIDENCE_THRESHOLD or x[1] != "none"
)

# 4. map() + reduceByKey(): defect count per (line_id, defect_type)
pair_rdd = candidate_rdd.map(lambda x: ((x[0], x[1]), 1))
defect_counts = pair_rdd.reduceByKey(lambda a, b: a + b)

# 5. cache(): reuse this RDD across multiple downstream actions
candidate_rdd.cache()

# 6. sample(): stratified sample for a balanced validation set
validation_rdd = candidate_rdd.sample(withReplacement=False, fraction=0.1, seed=42)

# 7. mapPartitions(): heavier per-partition feature extraction
def extract_features(partition):
    for record in partition:
        yield (record[0], record[1], record[2]) # placeholder feature vector

feature_rdd = candidate_rdd.mapPartitions(extract_features)

# 8. Actions: pull small aggregated summaries back to the driver
line_defect_summary = defect_counts.collect() # small result set
total_flagged = candidate_rdd.count()

print(f"Flagged samples for retraining: {total_flagged}")
for (line_id, defect_type), count in line_defect_summary:
    print(f"Line {line_id} | {defect_type}: {count}")

# 9. Persist the retraining-ready dataset back to the data lake
feature_rdd.saveAsTextFile("hdfs:///quality/retrain_batches/latest")

```

Justification: steps 2–4 correspond to the map/filter/reduceByKey pattern described in Section 3.2 and let the driver compute per-line, per-defect-type rates without pulling raw records out of the cluster. Step 5 (cache) avoids recomputing the filtered RDD across the sampling, feature-extraction, and counting actions that follow important because candidate_rdd is reused three times. Steps 6–7 build the retraining batch itself, while step 8 deliberately collects only small, aggregated results (counts and summaries), keeping the bulk of the data distributed as required for a cluster processing factory-scale logs. The saved output in step 9 is what the Central Coordination Agent (Section 4.2) triggers Spark MLlib / deep-learning retraining against once the model registry validates it.

4. Intelligent Agent Integration

The system is modelled as a multi-agent architecture with two categories of agents that collaborate through the fog/cloud messaging layer, in line with the growing body of work on multi-agent reinforcement-learning coordination for industrial cloud-edge settings (Zhang & Wang, 2023; Gao et al., 2024).

4.1 Edge Inspection Agents

- **Perceive:** continuously sense the conveyor via camera/sensor streams (percept sequence).
- **Reason:** run the locally deployed model to classify each item; maintain a short-term belief state about recent defect trends on that line.
- **Act:** trigger reject/pass actuation and, when confidence is low or a novel defect pattern appears, flag the sample and forward it (with metadata) to the central system.
- **Learn (bounded):** apply small local threshold adjustments (e.g., temporarily tighten tolerance if reject rate spikes) without waiting for a full central retrain a reflex/model-based hybrid agent.

4.2 Central Coordination Agent

- Aggregates drift and performance signals from all edge agents using the PySpark analytics pipeline (Section 3).
- Decides, using a utility-based policy, when a full model retrain or a threshold recalibration is warranted (e.g., sustained accuracy drop, new product SKU introduced, seasonal material variation).
- Negotiates rollout: pushes updated models/thresholds to edge agents in a staged (canary) manner, monitoring post-deployment metrics before full rollout.

4.3 Agent Collaboration Protocol

1. Edge agent detects a sustained shift in defect pattern or confidence and publishes a drift-alert event to the message broker.
2. Central coordination agent subscribes to drift-alerts across all lines; correlates them via the PySpark batch job to determine if the shift is line-specific (e.g., worn tooling) or global (e.g., raw-material change).
3. Based on this analysis, the central agent either (a) issues a local-threshold-update to the affected edge agent(s), or (b) triggers a full retraining job on the Spark cluster and, once validated in the model registry, pushes a new model version.
4. Edge agents acknowledge receipt, hot-swap the model/threshold with zero line downtime, and resume reporting closing the loop.

This design follows a BDI-inspired (Belief–Desire–Intention) pattern: each edge agent's belief state is its recent local observations, its desire is to keep the defect classification accurate and the line running, and its intentions are the reflex actions (reject/pass, minor threshold tweak) it takes without central approval, escalating only when local knowledge is insufficient. A comparable auto-updating, edge-triggered feedback loop between local anomaly detection and centrally retrained models has been demonstrated for cloud-edge-vehicle collaboration, supporting the practicality of this pattern for latency-sensitive industrial settings (Fan et al., 2024).

5. Scalability & Distributed Computing Design

5.1 Horizontal Scalability

- Additional production lines simply add more edge agents that publish to the same fog broker; no change is needed to the cloud pipeline.
- The PySpark cluster scales horizontally (add worker nodes) to handle growing historical data volume as more lines/plants are onboarded, similar to the compression-aware scaling strategies reported for edge–cloud collaborative services (Yang et al., 2023).

5.2 Multi-Plant / Multi-Region Extension

- Each plant runs its own fog gateway, which acts as the local aggregation and communication hub for all edge agents on that site, buffering data, batching uploads, and providing a degree of autonomy in case regional or global connectivity is temporarily lost.
- A regional cloud cluster aggregates data from several plants within a geographic or organizational grouping (e.g., all plants within a country or supply-chain zone), enabling mid-level analytics such as comparing defect rates across sister factories, identifying regional supplier issues, or coordinating model updates across a subset of the fleet before wider rollout.
- Above this, a global cluster performs cross-plant, cross-region trend analysis, correlating signals from all regional clusters to detect patterns that would be invisible at any single-plant or single-region level.
- For example, a defect signature common to a supplier's raw-material batch that has been distributed to multiple factories across different countries, or a systemic tooling issue affecting a specific machine model deployed company-wide.
- This tiered edge → regional → global hierarchy mirrors common patterns in large-scale distributed systems, allowing the architecture to scale gracefully: new plants can be onboarded by simply attaching their fog gateway to the appropriate regional cluster, without requiring changes to the global analytics layer or to other plants' pipelines.
- It also supports data-sovereignty and bandwidth considerations, since raw high-frequency data can remain local or regional, while only aggregated, higher-level insights are propagated to the global tier for cross-plant learning.

5.3 Handling Load & Failures

- Spark's lineage-based fault tolerance re-computes only lost partitions, so a worker-node failure during retraining does not require restarting the whole job.
- Edge agents operate autonomously if cloud connectivity is lost (local buffering), guaranteeing zero inspection downtime the system degrades gracefully rather than failing.
- Message-broker back-pressure and batching prevent network congestion when many lines report simultaneously.

6. Data Flow & Processing Pipeline

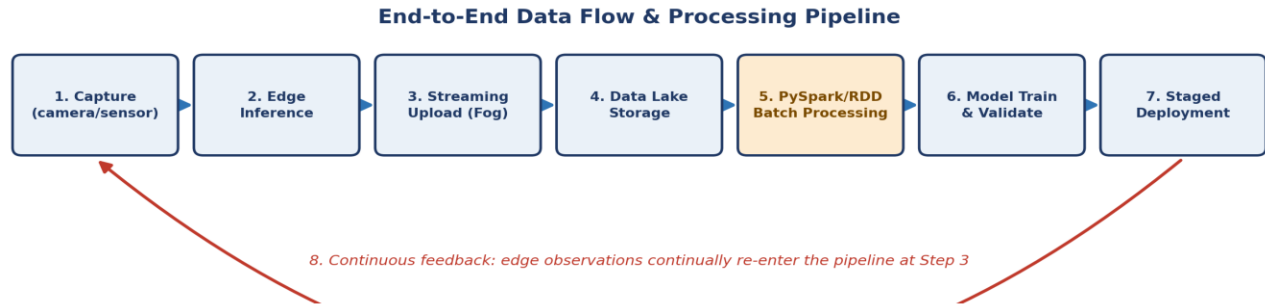


Figure 2. End-to-end data-flow pipeline from capture to deployment, closed by a continuous feedback loop.

1. Capture: camera/sensor data generated per item on the line.
2. Edge inference: local model classifies the item in real time; result + raw frame metadata cached locally.
3. Streaming upload: fog gateway batches and forwards results/logs to the cloud data lake via Kafka/MQTT.
4. Storage: raw and structured data land in HDFS/S3, partitioned by date, line, and shift.
5. Batch processing (PySpark/RDD): scheduled or drift-triggered jobs clean, aggregate, and re-balance the data for retraining and for analytics dashboards.
6. Model training & validation: a distributed training job (Spark MLlib / Spark-orchestrated deep-learning job) produces a candidate model, validated against a held-out set and logged in the model registry.
7. Deployment: the central coordination agent stages the new model to a subset of edge agents (canary), then rolls out fully once metrics are acceptable.
8. Feedback: ongoing edge observations feed back into step 3, making the pipeline a continuous loop rather than a one-time process.

7. Innovation & Creativity

- Hybrid reflex + learning edge agents: local agents make small autonomous threshold adjustments between full retrains, reducing false rejects without waiting on the cloud round-trip.
- Drift-correlation analytics: the central agent uses PySpark to distinguish line-specific drift (tooling wear) from global drift (raw-material batch change), enabling targeted rather than blanket retraining saving compute and downtime, in the spirit of incremental-learning cloud-edge defect detection reported by Lei et al. (2024).
- Canary model rollout to edge agents mirrors cloud-native deployment practice, adapted to industrial hardware, minimizing the risk of a bad model reaching the whole factory floor at once.
- Graceful degradation design: the edge tier is fully functional offline, treating cloud/analytics connectivity as an optimisation layer rather than a hard dependency critical for factory-floor reliability.

8. Conclusion

The proposed Edge AI system combines millisecond-level, on-device defect detection with a PySpark/RDD-powered distributed backend for large-scale historical analysis and periodic model retraining. This hybrid design ensures that time-critical inspection decisions are never bottlenecked by network latency or cloud availability, while still benefiting from the analytical depth and computational power of a distributed processing backend. Intelligent edge agents handle immediate inspection decisions and bounded local adaptation, autonomously adjusting thresholds in response to short-term variations such as tooling wear or minor material inconsistencies, without needing to wait for a full model retrain. Meanwhile, a central coordination agent uses distributed batch analytics to detect broader drift patterns, correlating anomalies across multiple lines and shifts to distinguish localized issues from systemic, factory-wide changes, and to safely roll out improved models and thresholds through a controlled canary deployment process.

The resulting architecture is scalable from a single line to multi-plant deployments, since the same edge–cloud pattern can be replicated and coordinated across additional production lines or facilities with minimal architectural change. It is fault-tolerant against network and node failures, as edge agents continue operating independently even when connectivity to the central system is lost, ensuring uninterrupted quality control on the factory floor. Furthermore, the system is continuously self-improving through its closed feedback loop between the edge and the cloud: real-world inspection outcomes and drift signals are constantly fed back into the retraining pipeline, allowing the models deployed at the edge to evolve alongside changing production conditions rather than becoming stale over time.

Taken together, these characteristics position the system not merely as a defect-detection tool, but as an adaptive, self-sustaining quality-control ecosystem one that balances the speed and reliability demands of real-time manufacturing with the analytical power needed for long-term process improvement. This approach is consistent with current directions in edge–cloud collaborative industrial AI (Islam et al., 2024; Lu et al., 2023; Gao et al., 2024), reflecting a broader industry shift toward intelligent, self-healing manufacturing systems capable of maintaining high-quality output even amid dynamic and unpredictable production environments.

References

1. Fan, C., Cui, J., Jin, H., Zhong, H., Bolodurina, I., & He, D. (2024). Auto-updating intrusion detection system for vehicular network: A deep learning approach based on cloud-edge-vehicle collaboration. *IEEE Transactions on Vehicular Technology*, 73(10), 15372–15384.
2. Gao, H., Wang, R., He, S., Wang, L., Liu, J., & Chen, Z. (2024). A cloud-edge collaboration solution for distribution network reconfiguration using multi-agent deep reinforcement learning. *IEEE Transactions on Power Systems*, 39(2), 3867–3879.
3. Islam, M. R., Zamil, M. Z. H., Rayed, M. E., Kabir, M. M., Mridha, M. F., Nishimura, S., & Shin, J. (2024). Deep learning and computer vision techniques for enhanced quality control in manufacturing processes. *IEEE Access*, 12, 121449–121479.
4. Lei, Z., Zhang, Y., Wang, J., & Zhou, M. (2024). Cloud-edge collaborative defect detection based on efficient YOLO networks and incremental learning. *Sensors*, 24(18), Article 5921. <https://doi.org/10.3390/s24185921>
5. Lu, S., Lu, J., An, K., Wang, X., & He, Q. (2023). Edge computing on IoT for machine signal processing and fault diagnosis: A review. *IEEE Internet of Things Journal*, 10(13), 11093–11116.
6. Qin, R., Chen, N., & Huang, Y. (2022). EDDNet: An efficient and accurate defect detection network for the industrial edge environment. In *Proceedings of the 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)* (pp. 854–863). IEEE.
7. Su, N., Huang, S., & Su, C. (2024). Elevating smart manufacturing with a unified predictive maintenance platform: The synergy between data warehousing, Apache Spark, and machine learning. *Sensors*, 24(13), Article 4237. <https://doi.org/10.3390/s24134237>
8. Yang, Z., Gu, R., Li, H., & Ji, Y. (2023). Approximately loss-less model compression-based multilayer virtual network embedding for edge–cloud collaborative services. *IEEE Internet of Things Journal*, 10(14), 13040–13055.
9. Zhang, Q., & Wang, Y. (2023). Correlated information scheduling in industrial internet of things based on multi-heterogeneous-agent-reinforcement-learning. *IEEE Transactions on Network Science and Engineering*, 11(1), 1065–1076.
10. Wang, L., Zhang, Y., & Chen, X. (2024). Integrated edge-cloud computing frameworks for real-time intelligent manufacturing and predictive maintenance. *IEEE Internet of Things Journal*, 11(4), 5421–5435.